

Generative Exploration via Stochastic Inference in Monte Carlo Tree Search for Automatic Heuristic Design

Anonymous Author(s)

Submission Id: «OpenReview submission id»

Abstract

Designing effective algorithmic components for NP-hard combinatorial optimization problems (COPs) remains a fundamental challenge, as traditional solvers predominantly rely on hand-crafted heuristics. While recent work uses large language models (LLMs) for automatic heuristic design (AHD), the resulting performance signals are highly noisy, as LLM-generated heuristics behave like samples from a high-variance stochastic process. Current approaches further reinforce this uncertainty by over-exploiting top-performing but fragile candidates. We propose GENeration Exploration via Stochastic Inference Monte Carlo Tree Search (GENESIS), an uncertainty-aware Monte Carlo Tree Search framework that models performance noise via Bayesian belief updating and integrates error-driven reflective reasoning. By explicitly modeling evaluation noise instead of averaging it away, GENESIS avoids over-committing to fragile candidates and surfaces robust ones from the same noisy LLM generator. Experiments on COP benchmarks show that GENESIS outperforms state-of-the-art LLM-AHD baselines in most problem instances under both ACO and construction framework settings. By transforming noisy trial-and-error into a structured search process, GENESIS offers a practical step toward fully automated algorithm design that requires less manual tuning and expert effort.

Keywords

Automatic Heuristic Design, Large Language Models, Monte Carlo Tree Search, Thompson Sampling, Combinatorial Optimization

ACM Reference Format:

Anonymous Author(s). 2027. Generative Exploration via Stochastic Inference in Monte Carlo Tree Search for Automatic Heuristic Design. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027)*, Hanoi, Vietnam, 3–7 May 2027, IFAAMAS, 8 pages.

1 Introduction

Combinatorial optimization problems (COPs), such as vehicle routing, scheduling, and coverage planning, are NP-hard and arise across logistics, manufacturing, and robotics [15, 23, 31, 35]. Due to their computational intractability, high-quality heuristics are essential for solving large-scale instances efficiently, yet designing such heuristics typically requires substantial domain expertise and manual tuning. This challenge has motivated hyper-heuristics (HH), also known as automatic heuristic design (AHD), which search over the space of heuristics rather than directly over the space of solutions [5, 6]. Earlier heuristic-generation approaches, often based

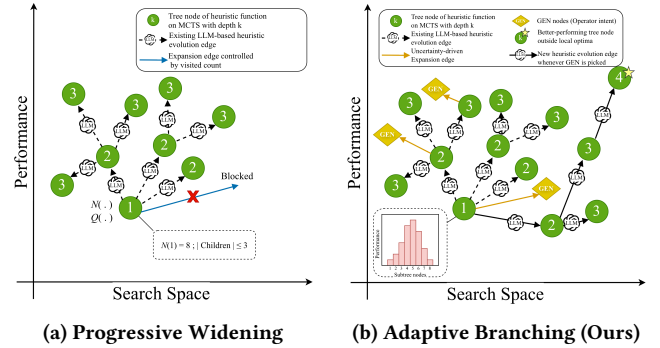


Figure 1: Conceptual comparison between (a) Progressive Widening and (b) Adaptive Branching in GENESIS. Progressive Widening limits tree expansion using a visit-count-based quota, resulting in uniform and blind branching. In contrast, GENESIS introduces GEN nodes representing operator intents and performs uncertainty-aware branching via Hierarchical Thompson Sampling, enabling targeted and adaptive expansion toward promising regions.

on genetic programming, automatically composed heuristics from predefined building blocks [13, 22]. More recently, large language models (LLMs) have opened a new direction for AHD by enabling heuristic synthesis directly from natural-language descriptions. This LLM-based paradigm expands the search space beyond manually specified operators and has been explored together with evolutionary search and tree-search methods, yielding competitive performance with limited human intervention [24, 39, 40].

Despite this promise, LLM-based AHD still faces fundamental limitations. LLM-generated heuristics are inherently stochastic and often highly sensitive to small prompt or semantic variations, so two generations from the same instruction can produce functionally different code with substantially different performance [29]. As a result, heuristic evaluation in this setting is noisy rather than deterministic. Existing methods often respond to this noise by aggressively favoring currently best-performing candidates, which can lead to over-exploitation of fragile heuristics and premature rejection of more robust structures. Therefore, the key challenge in LLM-based AHD is not only how to generate new heuristics, but also how to search under uncertainty when the underlying generator itself is unstable.

To address these limitations, we introduce GENESIS, which reformulates tree search for language hyper-heuristics by modeling node values as random variables, enabling Thompson sampling exploration, and incorporating error-driven reflection inspired by human thinking. We conduct extensive experiments across multiple COP benchmarks and heuristic design settings, and the results



This work is licensed under a Creative Commons Attribution International 4.0 License.

show that our framework outperforms strong LLM-based AHD baselines in most settings, highlighting the value of uncertainty-aware search and reflective refinement. Our contributions can be summarized as follows:

- We introduce a Bayesian Monte Carlo Tree Search (MCTS) framework that models node values as random variables with Normal–Inverse–Gamma priors and posteriors. By maintaining both global hyper-priors and local node posteriors, GENESIS explicitly captures evaluation uncertainty and reduces premature convergence to fragile heuristics.
- We propose a hierarchical Thompson sampling strategy for exploration-operator selection. By sampling from posterior beliefs at multiple levels of the search tree, GENESIS adaptively balances exploration and exploitation according to both uncertainty and observed performance.
- We introduce an error-driven reflection pipeline that simulates expert reasoning to refine heuristics during search through six sequential stages. This pipeline analyzes latent algorithmic weaknesses and counterfactual failures before generating revised code, thereby improving the robustness and generalizability of the discovered heuristics.

2 Related Work

2.1 Language Hyper-Heuristic

Recent approaches in automatic heuristic design leverage LLMs to generate, refine, or select heuristics from natural language prompts or high-level problem specifications. Methods such as FunSearch [33], AEL [25], EoH [24], ReEvo [39], HSEvo [9], MCTS-AHD [40] and PoH [38] combine evolutionary search or tree search with LLM-generated heuristics. By iteratively generating, executing, and refining programs, these language-driven hyper-heuristics can discover high-quality, adaptable heuristics with minimal domain knowledge.

2.2 Distributional Monte Carlo Tree Search

Distributional Monte Carlo Tree Search (MCTS) extends standard MCTS by modeling the full distribution of possible returns instead of only the expected value. This is particularly important in decision making under stochastic environment, where a single policy execution may have widely varying outcomes [1, 8, 17]. By maintaining a posterior over the utility of different returns, Distributional MCTS enables agents to make decisions that account for both risk and multiple objectives. Recent work has proposed a novel inference-time framework for LLMs that generalizes repeated sampling through principled multi-turn exploration and exploitation within tree search [19].

3 Preliminaries

3.1 Problem Definition

Definition 3.1 (Language Hyper-Heuristic (LHH)). Let S denote the space of problem instances of a combinatorial optimization problem (COP), and let H denote the space of heuristics for solving instances in S . Let $f : H \times S \rightarrow \mathbb{R}$ be a *solver evaluation function* or an *objective function*.

The *meta-objective* $F : H \rightarrow \mathbb{R}$ is defined as the average performance of a heuristic h over a dataset $\mathcal{J} \subseteq S$ of problem instances:

$$F(h) = \frac{1}{|\mathcal{J}|} \sum_{i \in \mathcal{J}} f(h, i).$$

Heuristics in H may belong to diverse algorithmic frameworks, and an optimal heuristic may be expressible in multiple ways. To simplify design, LHH methods focus on a *key heuristic function* with well-specified inputs and outputs; for example, in a step-by-step construction framework for the TSP, the key function selects the next city to visit given a partial tour. An LHH generates heuristics $h \in H$ probabilistically using a Large Language Model (LLM), conditioned on a seed heuristic $s_0 \in H$ and a natural-language prompt a :

$$h \sim \text{LLM}(s_0, a).$$

The optimal heuristic under this generative model is

$$h^* = \arg \min_h F(h).$$

Unlike traditional hyper-heuristics, which operate over a fixed, hand-crafted set of low-level heuristics, LHHs do not require a predefined heuristic space, thereby enabling exploration of an open-ended, linguistically grounded space of heuristics.

3.2 Bayes’ Theorem

Bayesian inference updates beliefs via Bayes’ rule:

$$p(\theta | D) \propto p(D | \theta)p(\theta).$$

For univariate Gaussian rewards $x \sim \mathcal{N}(\mu, \sigma^2)$ with unknown (μ, σ^2) , the conjugate prior is the *Normal–Inverse–Gamma* (NIG) distribution,

$$(\mu, \sigma^2) \sim \text{NIG}(\mu_0, \kappa_0, \alpha_0, \beta_0),$$

which factorizes as $\mathcal{N}(\mu | \mu_0, \sigma^2 / \kappa_0) \cdot \text{Inv-Gamma}(\sigma^2 | \alpha_0, \beta_0)$. Given data with mean $\bar{x}$ and sum of squares $S = \sum_i (x_i - \bar{x})^2$, the posterior remains NIG with

$$\begin{aligned} \kappa_n &= \kappa_0 + n, & \mu_n &= \frac{\kappa_0 \mu_0 + n \bar{x}}{\kappa_0 + n}, \\ \alpha_n &= \alpha_0 + \frac{n}{2}, & \beta_n &= \beta_0 + \frac{1}{2} \left(S + \frac{\kappa_0 n}{\kappa_0 + n} (\bar{x} - \mu_0)^2 \right). \end{aligned}$$

4 GENESIS Framework

4.1 Overall Architecture

GENESIS is an uncertainty-aware tree search framework for automatic heuristic design that combines adaptive Monte Carlo Tree Search with hierarchical Bayesian modeling (Figure 1). It explores an open-ended heuristic space by iteratively generating and evaluating heuristics using a fixed set of operators.

The search is organized as a dynamically growing tree with two types of nodes: *real nodes*, representing concrete heuristics with observed meta-objective values, and *operator nodes*, representing abstract transformation directions shared across the tree. Operator nodes do not correspond to directly evaluable solutions but aggregate evidence from heuristics generated by the same operator.

Each node is associated with a latent performance variable modeling the distribution of meta-objective values in its subtree, with uncertainty captured via Bayesian priors. Tree traversal is guided

by Thompson Sampling, enabling principled selection under uncertainty: selecting a real node continues traversal, while selecting an operator node triggers heuristic generation and expansion. Observed evaluations are propagated upward to update both local and shared uncertainty estimates.

GENESIS further incorporates an *Assumption-Guided Reflective Expansion (AGRE)* module, which analyzes high-performing heuristics to identify fragile assumptions and guide subsequent generation toward more robust designs.

4.2 Detailed Algorithm

4.2.1 Problem Setup. The search process is initialized with an empty root node s_0 . An initial set of N_0 heuristics is generated and added as real child nodes of s_0 , forming the first level of the search tree. Each node s in the search tree maintains two types of children: (i) a set of *real child nodes* $\mathcal{R}(s)$, each representing a concrete heuristic obtained by applying an operator, and (ii) a set of *operator nodes* $\mathcal{G}(s)$, with one operator node associated with each operator in the operator set $\mathcal{O} = \{o_1, \dots, o_K\}$. Operator nodes represent abstract transformation actions and do not correspond to directly evaluable heuristics.

For every node s in the search tree (real or operator or root), the algorithm maintains a local Bayesian posterior over a latent meta-objective parameter θ_s , modeled using a NIG distribution:

$$(\mu_s, \sigma_s^2) \sim \text{NIG}(\mu_0, \kappa_0, \nu_0, \tau_0^2). \quad (1)$$

Evaluation outcomes are assumed to follow

$$\theta_s \mid \mu_s, \sigma_s^2 \sim \mathcal{N}(\mu_s, \sigma_s^2). \quad (2)$$

The resulting posterior compactly summarizes all evaluation evidence collected for the corresponding heuristic or operator node. To enable statistical sharing across operator nodes, a global hyper-prior $\text{NIG}(\mu_{\text{global}}, \kappa_{\text{global}}, \nu_{\text{global}}, \tau_{\text{global}}^2)$ is defined and shared across all operator nodes in the tree. This hyper-prior governs the prior distribution of operator-level performance and is updated incrementally as new heuristics are generated and evaluated during the search.

4.2.2 Selection. The search process is initialized with a virtual root node s_0 . A set of initial heuristics with N_0 nodes is generated and added as real child nodes of s_0 . Starting from s_0 , the algorithm performs Thompson Sampling (TS) and Hierarchical Thompson Sampling (HTS) to guide tree traversal. At each state s , a global latent performance parameter (μ_0, σ_0^2) is first sampled from a shared global NIG hyper-posterior. This global sample is shared across all operator nodes $g \in \mathcal{G}(s)$, since they all rely on a common LLM to expand the tree. For each operator child node $g \in \mathcal{G}(s)$, a local posterior sample (μ_g, σ_g^2) is drawn from its NIG posterior. HTS then combines global and operator-level information via precision-weighted updating:

$$\begin{aligned} \lambda_g &= \frac{\kappa_g / \sigma_g^2}{\kappa_g / \sigma_g^2 + \kappa_0 / \sigma_0^2}, \\ \mu_g^{\text{comb}} &= \lambda_g \mu_g + (1 - \lambda_g) \mu_0, \\ \sigma_g^{2, \text{comb}} &= \left(\frac{\kappa_g}{\sigma_g^2} + \frac{\kappa_0}{\sigma_0^2} \right)^{-1}, \end{aligned}$$

$$\theta_g \sim \mathcal{N}(\mu_g^{\text{comb}}, \sigma_g^{2, \text{comb}}).$$

This hierarchical update jointly shrinks both the mean and variance of operator nodes toward the global belief, with the degree of shrinkage controlled by their relative posterior precision. In contrast, real child nodes $r \in \mathcal{R}(s)$ are treated as conditionally independent and are sampled solely from their local NIG posteriors:

$$(\mu_r^{\text{loc}}, \sigma_r^{2, \text{loc}}) \sim \text{NIG}_r, \quad \theta_r \sim \mathcal{N}(\mu_r^{\text{loc}}, \sigma_r^{2, \text{loc}}).$$

Although they may originate from the same operator, each real node represents a distinct heuristic instance. By maintaining separate uncertainty estimates, the algorithm avoids over-favoring unreliable heuristics during selection. The algorithm selects the node with the most favorable sampled value,

$$s^* \leftarrow \arg \min_{x \in \mathcal{G}(s) \cup \mathcal{R}(s)} \theta_x.$$

If a real node is selected, the selection process continues recursively from that node; otherwise, selecting an operator node terminates selection and triggers expansion.

4.2.3 Expansion. We design several LLM-based operators to generate new heuristics from existing ones. After a node is selected for expansion, the operator corresponding to the chosen operator node is applied to generate N new child heuristics, which are added to the search tree as real nodes.

E1 (Diversity-driven generation). Given multiple existing heuristics and their meta-objective values, the LLM generates a new heuristic that is structurally and algorithmically as different as possible from all provided ones, aiming to explore novel designs with lower meta-objective value.

E2 (Idea-guided recombination). Given several heuristics, the LLM first identifies common effective ideas from a reference heuristic and then designs a new heuristic inspired by these ideas while modifying the structure of another selected heuristic, with the goal of outperforming both parents.

M1 (Structural modification). Given a single heuristic, the LLM produces a modified version by introducing new mechanisms, equations, or program components, while preserving the core idea of the original algorithm.

M2 (Parameter variation). Given a single heuristic, the LLM generates a new variant by identifying key algorithmic parameters and altering their values or configurations, without changing the overall algorithmic structure.

S1 (Synthesis). Given multiple heuristics, the LLM extracts useful ideas from all of them and synthesizes a new heuristic that combines these insights to achieve better performance than any individual parent.

NS (Novelty Search). Given a heuristic, the LLM explicitly analyzes its weaknesses, inefficiencies, or limitations and constructs a new heuristic that exploits these vulnerabilities to counter or outperform the original approach.

4.2.4 Backpropagation. After expansion and evaluation, the observed meta-objective values of the newly generated children are propagated backward along the path $\mathcal{P}$ from the root s_0 . For each node $u \in \mathcal{P}$, the local NIG posterior is updated using the meta-objective values obtained from these newly generated nodes, thereby refining the node-specific uncertainty estimates.

In addition, whenever a node u corresponds to an operator node, its observations are also used to update the global NIG hyper-posterior. This global update aggregates evidence across all operator nodes generated by the same LLM-based transformation process, enabling principled statistical sharing while maintaining a conservative uncertainty estimate.

4.2.5 Assumption-Guided Reflective Expansion. Recent studies suggest that LLMs exhibit dual-process reasoning, combining fast intuitive responses with slower deliberative processes [7, 16]. While prior work primarily applies this paradigm to output-level refinement in question-answering, human experts also critically examine and revise the *assumptions* underlying solutions and algorithms.

Motivated by this observation, we propose *Assumption-Guided Reflective Expansion (AGRE)*, a structured pipeline that extends fast-slow reasoning to assumption-level analysis. AGRE proceeds through six stages, each corresponding to a distinct cognitive function in human expert reasoning.

Fast Intuitive Error Signaling. The LLM is prompted to generate brief, intuition-driven “discomfort signals” regarding the algorithm, highlighting brittle design choices, implicit assumptions, or potential silent failure modes. No proofs, fixes, or detailed explanations are allowed. This stage corresponds to fast, intuitive anomaly detection (System 1) [21].

Minimal Counterfactual Failure Construction. Conditioned on the detected error signals, the model constructs a single, minimal, and realistic scenario in which the algorithm fails due to a broken assumption. The emphasis is on plausibility rather than extremity, following classical theories of counterfactual reasoning in cognitive psychology [32]. This step transforms vague intuition into a concrete failure hypothesis.

Role-Based Reflective Conflict. The algorithm is examined through three explicitly separated roles: (i) a *Defender*, which justifies the original design under its intended assumptions; (ii) an *Attacker*, which argues that counterfactuals expose fundamental weaknesses; and (iii) an *Observer*, which synthesizes the remaining epistemic tension. This structured internal debate is inspired by the argumentative theory of reasoning, which holds that robust judgments emerge from adversarial evaluation of competing explanations [28]. It is further supported by recent work on adversarial prompting and debate-based LLM reasoning, showing that explicit role opposition and self-critique improve factuality and error detection [12, 20, 27].

Strategic Abstraction. The analysis is lifted from implementation details to a higher level of abstraction, focusing on the algorithm’s core strategy, trade-offs, and assumption structure. This abstraction step prevents overfitting to surface-level artifacts and supports transferable insight across problem instances.

Assumption Repair via Metacognitive Regulation. Rather than modifying the algorithm itself, AGRE refines or restricts only the algorithm’s assumptions or applicability conditions. This mirrors metacognitive regulation in human problem solving, where robustness is improved by clarifying when a heuristic should or should not be trusted [14].

Assumption-Conditioned Refinement Guidance. Finally, the repaired assumptions are translated into concrete, implementable

guidance for improving robustness, stability, or generalization, while preserving the original algorithmic intent.

After generating N candidate child heuristics, the child with the lowest meta-objective value is selected for AGRE. The resulting heuristic is appended to the search tree, and its evaluation outcome is propagated backward along the path to the root.

5 Experiments

5.1 Experimental Setting

Language Model. All experiments are conducted using *codestral-2508*. This model is accessed via the official Mistral API using publicly available (free-tier) API keys. Despite the absence of paid or proprietary fine-tuning, Codestral demonstrates strong code generation and program synthesis capabilities.

Benchmark. We compare GENESIS with other LHH baselines within an Ant Colony Optimization (ACO) framework. All methods share the same underlying ACO construction process and differ only in how heuristics or operators are generated and selected. The comparison spans four combinatorial optimization problems: Traveling Salesman Problem (TSP), Multiple Knapsack Problem (MKP), Capacitated Vehicle Routing Problem (CVRP), and Offline Bin Packing Problem (Offline BPP). For TSP and KP, both offline heuristic generation and step-by-step solution construction settings are considered.

Setting. All experiments are conducted on a single Intel Core i7-10850H CPU. For all problem instances, the maximum wall-clock time for heuristic design is limited to 60s. For ACO-based methods, the maximum number of function evaluations is set to 500, while for step-by-step constructive methods, the evaluation budget is increased to 1,000. The initial heuristic pool size is set to $N_0 = 5$. During tree expansion, each operator node generates $N = 4$ new candidate heuristics. All methods share the same evaluation interface and objective functions. For each LLM-based AHD baseline, we perform three independent runs and report the average performance, while selecting the best result among the three runs for comparison.

5.2 Results

Ant Colony Optimization (ACO). Ant Colony Optimization (ACO) is a population-based metaheuristic inspired by the foraging behavior of ants, originally introduced as the Ant System [11] for solving combinatorial optimization problems. In ACO, artificial ants iteratively construct solutions by traversing a problem graph according to probabilistic transition rules that balance pheromone intensity and heuristic information. Pheromone trails are updated based on solution quality, reinforcing promising solution components over time. The Ant Colony System (ACS) further improves this framework through local pheromone updates and elitist reinforcement strategies [10]. Within this paradigm, the heuristic matrix plays a central role in guiding path selection by encoding problem-specific instance information. By parameterizing or generating this heuristic matrix, external models can directly influence the solution construction process, effectively transforming ACO into a general-purpose optimization framework. In this work, we leverage LLM-based automatic heuristic design (AHD) to generate

Table 1: Performance comparison on combinatorial optimization problems under the ACO framework. Results report average objective values ($\pm$ standard deviation) over 3 runs; the evaluation dataset consists of 64 instances for each problem and instance size. Lower is better ($\downarrow$) for TSP, CVRP, and BPP; higher is better ($\uparrow$) for MKP. Best results are in bold.

Problem	Size	EoH	ReEvo	HSEvo	MCTS-AHD	GENESIS
TSP $\downarrow$	20	3.867 ± 0.002	3.878 ± 0.010	3.874 ± 0.011	3.878 ± 0.010	3.866 ± 0.003
	50	5.860 ± 0.013	5.862 ± 0.003	5.871 ± 0.022	5.862 ± 0.003	5.807 ± 0.076
	100	8.453 ± 0.028	8.442 ± 0.079	8.388 ± 0.029	8.442 ± 0.079	8.385 ± 0.133
CVRP $\downarrow$	20	4.919 ± 0.056	5.013 ± 0.185	4.931 ± 0.193	4.959 ± 0.079	4.842 ± 0.032
	50	9.588 ± 0.163	9.595 ± 0.311	9.629 ± 0.247	9.503 ± 0.063	9.004 ± 0.347
	100	17.039 ± 0.476	16.503 ± 0.992	16.868 ± 0.426	16.544 ± 0.170	15.827 ± 0.244
MKP $\uparrow$	100	16.850 ± 0.158	17.013 ± 0.034	16.945 ± 0.065	17.066 ± 0.017	17.048 ± 0.037
	300	51.936 ± 0.768	53.556 ± 0.662	46.793 ± 8.028	53.222 ± 1.308	53.993 ± 0.191
	500	124.067 ± 1.383	130.687 ± 1.771	128.331 ± 3.399	129.781 ± 1.099	131.044 ± 0.948
BPP $\downarrow$	120	49.531 ± 0.095	49.828 ± 0.053	49.661 ± 0.307	49.682 ± 0.180	49.453 ± 0.041
	500	205.115 ± 0.070	206.505 ± 0.503	205.474 ± 0.609	205.177 ± 0.095	204.865 ± 0.048
	1000	409.208 ± 0.079	412.182 ± 1.340	410.271 ± 1.220	408.984 ± 0.688	408.719 ± 0.016

heuristic functions, enabling ACO to be flexibly applied across a diverse set of combinatorial optimization problems. Table 1 compares the performance of GENESIS with other baseline LHHs. The test dataset consists of 64 instances for every problem and instance size.

Step-by-Step Construction Framework. We evaluate GENESIS against competing LHH frameworks under the *step-by-step construction* paradigm, where feasible solutions are incrementally assembled by sequentially selecting components conditioned on the current partial state [3, 4]. This formulation generalizes naturally across routing, packing, and synthetic combinatorial tasks. Specifically, we consider: (i) the Traveling Salesman Problem (TSP), where cities are iteratively appended to form a complete tour; (ii) the Knapsack Problem (KP), where items are selected until the capacity constraint prevents further insertions and performance is reported as optimality gap; and (iii) the Admissible Sets Problem (ASP), a synthetic maximization task over ternary vectors subject to triple-wise admissibility constraints. Consolidated performance results across all problem types and instance sizes are presented in Table 2.

Guided Local Search Framework. Guided Local Search (GLS) is a metaheuristic framework that improves local search by dynamically penalizing solution features responsible for poor local optima. Originally proposed by [36] and later extended for the Traveling Salesman Problem in [37], GLS works by augmenting the original objective with adaptive penalties. When the local search reaches a local minimum, GLS increases the penalties of selected features with high utility, thereby modifying the search landscape and encouraging exploration of new regions.

We employ LLM-based AHD methods to design the key heuristic function for constructing knowledge-based matrices within the knowledge-guided local search (KGLS) framework [2]. KGLS maintains both the current solution and the best solution encountered so far throughout the search process. At each iteration, the current solution is first perturbed according to the generated knowledge matrix, followed by a local improvement phase using 2-opt and relocate operators [34].

In our experiments, we employ KGLS solely for solution construction on large-scale TSP instances. The dataset consists of 64 instances for each problem size. The optimal solutions are obtained using LKH-3 [18]. Results are reported in Table 3.

5.3 Analysis on Meta-objective Distribution

Further, we investigate the distribution of meta-objective values discovered by each framework on each problem. For each framework–problem pair, we aggregate results from multiple independent runs to obtain approximately 1,000 samples, enabling a statistically meaningful comparison of their optimization behaviors.

The objective landscape of LLM-based automatic heuristic design is highly multimodal and rugged [26]. As evidenced by Tables 4 and 5, GENESIS achieves the lowest minimum objective value on both TSP and CVRP ACO, which is consistent with the additional exploitation provided by AGRE around promising heuristics. At the same time, GENESIS has the highest mean objective value on both problems, suggesting that it spends a larger share of its budget on less promising regions instead of concentrating all evaluations around the incumbent, in line with its uncertainty-driven operator selection. GENESIS also exhibits the lowest skewness among all methods, i.e., its distribution has a less pronounced right tail relative to its spread. Taken together, these statistics are consistent with a balance between exploitation and exploration; however, they describe the distribution of generated heuristics and do not by themselves establish the causal effect of individual components.

5.4 Ablation Study

5.4.1 Discussion on the Effect of AGRE. By augmenting HSEvo with AGRE, we assess whether AGRE leads to increased exploitation within another framework. After selecting the best individuals from the population at each generation, we apply AGRE to them. Compared with the original HSEvo results in Table 1, AGRE improves HSEvo in three of the four settings (all except TSP-100), as shown in Table 6, suggesting that AGRE is not specific to our search framework. Nevertheless, HSEvo + AGRE remains behind

Table 2: Performance comparison on constructive combinatorial optimization problems. Results report average values ($\pm$ standard deviation) over 3 runs. Evaluation: 64 instances per TSP size (tour length); 1,000 instances per KP size with capacity 25 (optimality gap in %, optimal values computed using Google OR-Tools [30]); ASP instances are given as (n, w) . Lower is better ($\downarrow$) for TSP and KP; higher is better ($\uparrow$) for ASP. Best results are in bold.

Problem	Size	EoH	ReEvo	HSEvo	MCTS-AHD	GENESIS
TSP $\downarrow$	20	4.163 $\pm$ 0.101	4.073 $\pm$ 0.022	4.049 $\pm$ 0.027	4.056 $\pm$ 0.018	4.030 $\pm$ 0.016
	50	6.649 $\pm$ 0.259	6.367 $\pm$ 0.018	6.329 $\pm$ 0.033	6.635 $\pm$ 0.718	6.304 $\pm$ 0.054
	100	9.258 $\pm$ 0.472	8.697 $\pm$ 0.032	8.654 $\pm$ 0.020	8.671 $\pm$ 0.026	8.623 $\pm$ 0.126
	200	13.116 $\pm$ 0.684	12.231 $\pm$ 0.030	12.165 $\pm$ 0.093	12.145 $\pm$ 0.050	12.040 $\pm$ 0.182
KP $\downarrow$	100	0.080 $\pm$ 0.017	0.091 $\pm$ 0.003	0.074 $\pm$ 0.009	0.092 $\pm$ 0.005	0.073 $\pm$ 0.007
	200	0.082 $\pm$ 0.013	0.105 $\pm$ 0.002	0.089 $\pm$ 0.006	0.104 $\pm$ 0.002	0.084 $\pm$ 0.015
	500	0.057 $\pm$ 0.014	0.058 $\pm$ 0.011	0.062 $\pm$ 0.012	0.089 $\pm$ 0.027	0.051 $\pm$ 0.015
	1000	0.053 $\pm$ 0.031	0.044 $\pm$ 0.013	0.044 $\pm$ 0.014	0.088 $\pm$ 0.038	0.037 $\pm$ 0.012
ASP $\uparrow$	(12,7)	783.00 $\pm$ 15.59	783.00 $\pm$ 15.59	792.00 $\pm$ 0.00	792.00 $\pm$ 0.00	792.00 $\pm$ 0.00
	(15,10)	12300.33 $\pm$ 2014.45	10230.00 $\pm$ 2195.75	9945.33 $\pm$ 3972.60	12082.33 $\pm$ 940.79	13559.67 $\pm$ 393.56
	(21,15)	26501.67 $\pm$ 4047.10	21678.00 $\pm$ 4388.37	20018.33 $\pm$ 6985.57	23679.67 $\pm$ 6718.00	32211.67 $\pm$ 893.39
	(24,17)	119749.00 $\pm$ 26711.08	92756.00 $\pm$ 23062.51	84125.00 $\pm$ 33275.70	95989.00 $\pm$ 25072.55	152286.00 $\pm$ 15091.02

Table 3: Average optimality gap (%) over 3 runs for LHH frameworks on large-scale TSP instances under the KGLS framework. Evaluation is performed on 64 instances per size with numbers of cities {500, 700, 1000, 1500}. Lower values are better ($\downarrow$).

Method	TSP500 $\downarrow$	TSP700 $\downarrow$	TSP1000 $\downarrow$	TSP1500 $\downarrow$
ReEvo	0.979 $\pm$ 0.046	1.277 $\pm$ 0.011	1.601 $\pm$ 0.009	1.965 $\pm$ 0.022
HSEvo	0.930 $\pm$ 0.026	1.263 $\pm$ 0.052	1.593 $\pm$ 0.034	1.958 $\pm$ 0.095
MCTS-AHD	0.930 $\pm$ 0.022	1.237 $\pm$ 0.024	1.563 $\pm$ 0.047	1.903 $\pm$ 0.055
GENESIS	0.909 $\pm$ 0.032	1.206 $\pm$ 0.047	1.467 $\pm$ 0.089	1.850 $\pm$ 0.083

Table 4: Meta-objective distribution statistics on TSP ACO.

Method	Min	P1 (1%)	P5 (5%)	Mean	Std	Skewness
EoH	5.807	5.834	5.855	9.435	6.498	1.893
ReEvo	5.784	5.815	5.833	7.067	3.393	4.151
MCTS-AHD	5.814	5.835	5.865	8.582	5.161	2.424
GENESIS	5.710	5.720	5.851	10.714	6.081	1.136

Table 5: Meta-objective distribution statistics on CVRP ACO.

Method	Min	P1 (1%)	P5 (5%)	Mean	Std	Skewness
EoH	9.298	9.419	9.534	13.310	5.392	2.097
ReEvo	9.212	9.416	9.567	14.646	6.712	1.686
MCTS-AHD	9.096	9.132	9.216	11.920	4.069	2.443
GENESIS	8.920	9.277	9.490	14.818	4.975	0.720

the full GENESIS in all four settings and behind GENESIS without AGRE in three of them (Table 7), indicating that the search strategy itself also contributes to the performance of GENESIS.

5.4.2 Ablation on Parameters and Components. Since GENESIS combines several components, we disentangle their individual contributions by removing or replacing one component at a time while keeping all other settings (LLM, evaluation budget, N_0 , N) fixed. Results are reported in Table 7. The search-level variants are defined as follows:

Table 6: Performance of HSEvo augmented with AGRE compared with GENESIS on TSP ACO and CVRP ACO (Size 50, 100). Lower is better ($\downarrow$).

Method	TSP-50 $\downarrow$	TSP-100 $\downarrow$	CVRP-50 $\downarrow$	CVRP-100 $\downarrow$
HSEvo + AGRE	5.855 $\pm$ 0.007	8.406 $\pm$ 0.086	9.242 $\pm$ 0.072	15.854 $\pm$ 0.305
GENESIS	5.807 $\pm$ 0.076	8.385 $\pm$ 0.133	9.004 $\pm$ 0.347	15.827 $\pm$ 0.244

- **w/o posterior sampling:** Thompson sampling is replaced by a deterministic UCB rule computed on the posterior means, so that node selection no longer accounts for the full posterior uncertainty.
- **w/o adaptive op. selection:** once a real node is selected, the operator is drawn uniformly at random from $\mathcal{O}$ instead of being chosen by sampling from the operator-node posteriors.
- **w/o global sharing:** the global NIG hyper-prior is removed, i.e., operator nodes are sampled solely from their local posteriors ($\lambda_g = 1$), so no statistical strength is shared across operator nodes.
- **w/o AGRE:** the reflective expansion module is disabled and the generated children are added to the tree directly.

Removing the entire AGRE module (w/o AGRE) degrades performance in three of the four settings, most visibly on CVRP (9.202 vs. 9.004 on CVRP-50 and 15.947 vs. 15.827 on CVRP-100), whereas on TSP-100 the variant without AGRE is slightly better (8.319 vs. 8.385). Given the standard deviations over three runs, these differences should be read as trends rather than statistically significant effects.

To further analyze the internal design of AGRE, we individually remove the Counterfactual Failure analysis, the Role-Based Reflection mechanism, and the Assumption Repair mechanism. Removing any single stage degrades performance on TSP-50, and removing Role-Based Reflection also degrades both CVRP settings. On TSP-100, removing Counterfactual Failure hurts performance, whereas removing Role-Based Reflection or Assumption Repair yields

slightly better results. Overall, the full AGRE pipeline performs best in most of the evaluated settings, but the contribution of each individual stage is modest.

Table 7: Ablation study of GENESIS components on TSP ACO and CVRP ACO (Size 50, 100). Each row removes or replaces a single component of the full model. CF: Counterfactual Failure; RR: Role-Based Reflection; AR: Assumption Repair. Lower is better (↓).

Method	TSP-50 ↓	TSP-100 ↓	CVRP-50 ↓	CVRP-100 ↓
<i>Search components</i>				
w/o posterior sampling	–	–	–	–
w/o adaptive op. selection	–	–	–	–
w/o global sharing	–	–	–	–
w/o AGRE	5.835 ± 0.024	8.319 ± 0.078	9.202 ± 0.063	15.947 ± 0.088
<i>AGRE stages</i>				
AGRE w/o CF	5.881 ± 0.010	8.469 ± 0.108	–	–
AGRE w/o RR	5.876 ± 0.129	8.342 ± 0.068	9.308 ± 0.211	16.019 ± 0.092
AGRE w/o AR	5.849 ± 0.030	8.369 ± 0.143	–	–
GENESIS	5.807 ± 0.076	8.385 ± 0.133	9.004 ± 0.347	15.827 ± 0.244

References

- [1] Anonymous. 2025. Distributional Monte-Carlo Tree Search with Thompson Sampling in Stochastic Environments. Submitted to The Fourteenth International Conference on Learning Representations. under review. <https://openreview.net/forum?id=QNHJSRO8xE>
- [2] Florian Arnold and Kenneth Sørensen. 2019. Knowledge-guided local search for the vehicle routing problem. *Computers & Operations Research* 105 (2019), 32–46. doi:10.1016/j.cor.2019.01.002
- [3] Emmanuel O. Asani, Aderemi E. Okeyinka, and Ayodele Ariyo Adebiji. 2023. A Computation Investigation of the Impact of Convex Hull Subtours on the Near-est Neighbor Heuristic. In *Proceedings of the 2023 International Conference on Science, Engineering and Business for Sustainable Development Goals (SEB-SDG)*. 1–7. doi:10.1109/SEB-SDG57117.2023.10124469
- [4] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, and Samy Bengio. 2017. Neural Combinatorial Optimization with Reinforcement Learning. arXiv:1611.09940 [cs.AI] <https://arxiv.org/abs/1611.09940>
- [5] Edmund K. Burke, Michel Gendreau, Matthew Hyde, Graham Kendall, Gabriela Ochoa, Ender Ozcan, and Rong Qu. 2013. Hyper-Heuristics: A Survey of the State of the Art. *Journal of the Operational Research Society* 64, 12 (2013), 1695–1724.
- [6] Edmund K. Burke, Matthew Hyde, Graham Kendall, Gabriela Ochoa, Ender Ozcan, and John R. Woodward. 2010. A Classification of Hyper-Heuristic Approaches. In *Handbook of Metaheuristics*, Michel Gendreau and Jean-Yves Potvin (Eds.). Springer, 449–468.
- [7] Stephen Chung, Wenyu Du, and Jie Fu. 2025. Thinker: Learning to Think Fast and Slow. arXiv:2505.21097 [cs.CL] <https://arxiv.org/abs/2505.21097>
- [8] Tuan Dam, Pascal Stenger, Lukas Schneider, Joni Pajarinen, Carlo D’Eramo, and Odalric-Ambrym Maillard. 2023. Monte-Carlo tree search with uncertainty propagation via optimal transport. arXiv:2309.10737 [cs.AI] <https://arxiv.org/abs/2309.10737>
- [9] Pham Vu Tuan Dat, Long Doan, and Huynh Thi Thanh Binh. 2025. HSEvo: Evaluating automatic heuristic design with diversity-driven harmony search and genetic algorithm using LLMs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 26931–26938. <https://github.com/datphamvn/HSEvo>
- [10] Marco Dorigo and Luca M. Gambardella. 1997. Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation* 1, 1 (1997), 53–66.
- [11] Marco Dorigo, Vittorio Maniezzo, and Alberto Colnani. 1996. The ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B* 26, 1 (1996), 29–41.
- [12] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving Factuality and Reasoning in Language Models through Multiagent Debate. arXiv:2305.14325 [cs.CL] <https://arxiv.org/abs/2305.14325>
- [13] Michael G. Epitropakis and Edmund K. Burke. 2025. Hyper-Heuristics. In *Handbook of Heuristics*, Rafael Martí, Panos M. Pardalos, and Mauricio G. C. Resende (Eds.). Springer, 687–743.
- [14] John H. Flavell. 1979. Metacognition and Cognitive Monitoring: A New Area of Cognitive–Developmental Inquiry. *American Psychologist* 34, 10 (1979), 906–911. doi:10.1037/0003-066X.34.10.906
- [15] Michael R. Garey and David S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- [16] Thilo Hagendorff, Sarah Fabi, and Michal Kosinski. 2023. Human-like intuitive behavior and reasoning biases emerged in large language models but disappeared in ChatGPT. *Nature Computational Science* 3, 10 (Oct. 2023), 833–838. doi:10.1038/s43588-023-00527-x
- [17] Conor F. Hayes, Mathieu Reymond, Diederik M. Roijers, Enda Howley, and Patrick Mannion. 2021. Risk Aware and Multi-Objective Decision Making with Distributional Monte Carlo Tree Search. arXiv:2102.00966 [cs.LG] <https://arxiv.org/abs/2102.00966>
- [18] Keld Helsgaun. 2017. *An Extension of the Lin-Kernighan-Helsgaun TSP Solver for Constrained Traveling Salesman and Vehicle Routing Problems*. Technical Report. Department of Computer Science, Roskilde University, Roskilde, Denmark.
- [19] Yuichi Inoue, Kou Misaki, Yuki Imajuku, So Kuroki, Taishi Nakamura, and Takuya Akiba. 2025. Wider or Deeper? Scaling LLM Inference-Time Compute with Adaptive Branching Tree Search. arXiv:2503.04412 [cs.AI] <https://arxiv.org/abs/2503.04412>
- [20] Geoffrey Irving, Paul Christiano, and Dario Amodei. 2018. AI safety via debate. arXiv:1805.00899 [stat.ML] <https://arxiv.org/abs/1805.00899>
- [21] Daniel Kahneman. 2011. *Thinking, Fast and Slow*. Farrar, Straus and Giroux.
- [22] William B. Langdon and Riccardo Poli. 2002. *Foundations of Genetic Programming*. Springer, Berlin, Heidelberg.
- [23] Gilbert Laporte. 2009. Fifty Years of Vehicle Routing. *Transportation Science* 43, 4 (2009), 408–416.
- [24] Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. 2024. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. In *International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2401.02051>
- [25] Fei Liu, Xialiang Tong, Mingxuan Yuan, and Qingfu Zhang. 2023. Algorithm Evolution Using Large Language Model. arXiv:2311.15249 [cs.NE] <https://arxiv.org/abs/2311.15249>
- [26] Fei Liu, Qingfu Zhang, Jialong Shi, Xialiang Tong, Kun Mao, and Mingxuan Yuan. 2025. Fitness Landscape of Large Language Model-Assisted Automated Algorithm Search. arXiv:2504.19636 [cs.AI] <https://arxiv.org/abs/2504.19636>
- [27] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651 [cs.CL] <https://arxiv.org/abs/2303.17651>
- [28] Hugo Mercier and Dan Sperber. 2011. Why do humans reason? Arguments for an argumentative theory. *Behavioral and Brain Sciences* 34, 2 (2011), 57–74.
- [29] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. 2025. An Empirical Study of the Non-Determinism of ChatGPT in Code Generation. *ACM Transactions on Software Engineering and Methodology* 34, 2 (Jan. 2025), 1–28. doi:10.1145/3697010
- [30] Laurent Perron and Vincent Furnon. 2024. OR-Tools. Version 9.10, released 2024-05-07. <https://developers.google.com/optimization/>
- [31] Michael Pinedo. 2016. *Scheduling: Theory, Algorithms, and Systems*. Springer.
- [32] Neal J. Roese. 1997. Counterfactual thinking. *Psychological Bulletin* 121, 1 (1997), 133–148.
- [33] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2023. Mathematical discoveries from program search with large language models. *Nature* (2023). doi:10.1038/s41586-023-06924-6
- [34] Lahari Sengupta, Radu Marinescu-Istodor, and Pasi Fränti. 2019. Which local search operator works best for the open-loop TSP? *Applied Sciences* 9, 19 (2019), 3985.
- [35] Chee Sheng Tan, Rosmiwati Mohd-Mokhtar, and Mohd Rizal Arshad. 2021. A Comprehensive Review of Coverage Path Planning in Robotics Using Classical and Heuristic Algorithms. *IEEE Access* 9 (2021), 119310–119342.
- [36] C. Voudouris and Edward Tsang. 1996. Partial constraint satisfaction problems and guided local search. In *Proceedings of the Second International Conference on the Practical Application of Constraint Technology (PACT)*.
- [37] Christos Voudouris and Edward Tsang. 1999. Guided local search and its application to the traveling salesman problem. *European Journal of Operational Research* 113, 2 (1999), 469–499.
- [38] Hui Wang, Xufeng Zhang, and Chaoxu Mu. 2025. Planning of Heuristics: Strategic Planning on Large Language Models with Monte Carlo Tree Search for Automating Heuristic Optimization. arXiv:2502.11422 [cs.AI] <https://arxiv.org/abs/2502.11422>
- [39] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. 2024. ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution. In *Advances in Neural Information Processing Systems*. <https://github.com/ai4co/reveo>
- [40] Zhi Zheng, Zhuoliang Xie, Zhenkun Wang, and Bryan Hooi. 2025. Monte Carlo Tree Search for Comprehensive Exploration in LLM-Based Automatic Heuristic Design. In *Proceedings of the International Conference on Machine Learning*

(ICML). <https://arxiv.org/abs/2501.08603>